

Java Basic

@author Yangxin Wu

Last edited: 20251125

- [常量与变量](#)
 - [常量](#)
 - [变量](#)
- [输入与输出](#)
 - [输入](#)
 - [输出](#)
- [数组](#)
 - [声明和创建](#)
 - [长度和索引](#)
 - [引用类型](#)
 - [多维数组](#)
- [异常](#)
 - [捕获异常](#)
 - [自定义异常](#)
- [虚拟机](#)
- [类与对象](#)
 - [对象](#)
 - [类](#)
 - [类的构造](#)
 - [静态类](#)
- [继承](#)
 - [超类](#)
 - [抽象类与接口](#)
 - [多重继承](#)

Java 和 C/C++ 在很多语法上相似甚至完全一致，因此这里只列出 Java 与 C 不同的语言特性。

常量与变量

常量

- 布尔常量: `true`, `false`
- 整数常量: 八进制 (`015`)、十进制 (`13`)、十六进制 (`0xD`)
- 浮点常量

- 单精度，用 `f` 做后缀，如 `13.5f`
- 双精度，用 `d` 做后缀
- 无后缀默认为双精度
- 字符常量
- 字符串常量
- 符号常量：用标识符表示的常量，要先声明后使用。
 - `[修饰符]` 类型标识符 `final` 常量名 = (直接) 常量；
 - 修饰符是表示该常量适用范围的权限修饰符，常用的有 `public`, `private`, `protected`
 - `final` 是定义常量的关键字，表示不能改变。
 - 类型标识符：静态常量 (`static`)、成员常量和局部常量

变量

程序运行过程中其值可以改变的量。

```
[修饰符] 类型标识符 常量名 = 常量；
```

输入与输出

输入

Java 的标准输入流 `System.in` 是一个 `InputStream` 对象，有两种常用的读取方式：

- 用读字符流的方法将 `System.in` 读到一个 `BufferedReader` 中。

```
BufferedReader stdin = new BufferedReader(new  
InputStreamReader(System.in));  
System.out.println(stdin.readLine());
```

- 用 `java.util.Scanner` 对 `System.in` 进行封装。

```
Scanner stdin = new Scanner(System.in);  
System.out.println(stdin.nextLine());
```

输出

Java 的标准输出流 `System.out` 是一个 `PrintStream` 对象。

- `System.out.print()` 不换行输出。
- `System.out.println()` 换行输出。
- `System.out.printf()` 格式化输出，语法同 C 标准库的 `printf()`。
- `System.out.write()` 输出字节数组，在输出时不换行。

这里列出几种 `write()` 的用法：

```
// Write a single byte.  
System.out.write(65); // 65 (ASCII) -> 'A'  
  
// Write a byte array.  
byte[] data = "Hello Java\n".getBytes(StandardCharsets.UTF_8);  
System.out.write(data); // 'Hello Java\n'  
  
// Write part of a byte array.  
byte[] data = "abcdef".getBytes();  
System.out.write(data, 1, 3); // 'bcd'
```

再使用 `write()` 后，需要用 `flush()` 方法清理缓冲区：

```
System.out.flush();
```

数组

Java 中的「数组」是确定长度的，一经创建就不能再加长了。

声明和创建

有两种声明数组的方式，第一种是 Java 语言规范提倡的方式：

```
int[] a;
```

第二种是 C 语言风格声明方式：

```
int b[];
```

在创建数组时，要么指定长度：

```
int[] a = new int[3];
```

要么给出所有的项：

```
double[] b = new double[]{ 2.1, 4.5, 4.2 };
```

长度和索引

可以使用 `arr.length` 得到一个数组的长度。数组的下标从 0 开始，到 `arr.length - 1` 结束。

引用类型

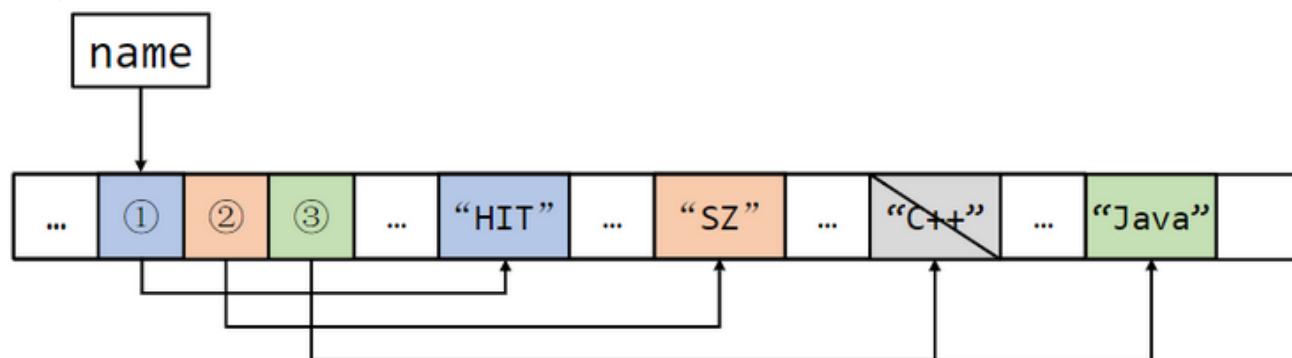
数组是「引用类型」，可以理解为 C 家族中的「指针」。下面的代码解释了「引用」的本质。

```
int[] arr = new int[] { 1, 4, 2, 5 };
System.out.println(arr[2]);           // 2
arr = new int[] { 7, 3, 6, 9 };       // arr 指向了新数组，旧数组会被 JVM 回收
System.out.println(arr[2]);           // 6
```

思考下面代码的输出：

```
String[] name = { "HIT", "SZ", "C++" };
String s = name[2];
name[2] = "Java";
System.out.println(name[2]); // 'Java'
System.out.println(s);      // 'C++'
```

内存映射图如下：



多维数组

本质上是「数组的数组」。例如

```
int[][] arr = new int[5][];
```

这个二维数组内部的 5 个数组因为现在还没有初始化，所以长度可以不给出。

我们当然也可以在定义给出所有的项，如下：

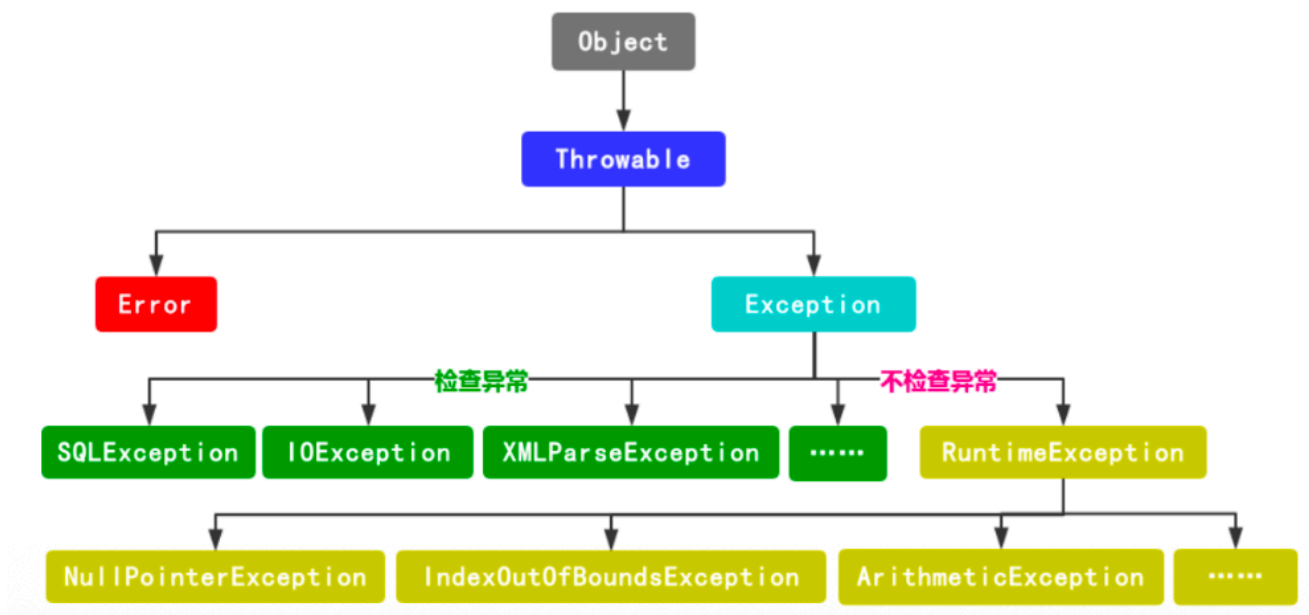
```
int[][] arr = new int[][] { { 1, 4, 2 }, { 2, 1 }, { 9, 1, 5, 6 } };
```

显然，多维数组中的每一行不一定要一样长。

异常

程序运行时发生的不被期望的事件，阻止了程序按照程序员的预期正常执行。

- 检查性异常：最具代表的检查性异常是用户错误或问题引起的异常，这是程序员无法预见的。例如要打开一个不存在文件时，一个异常就发生了，这些异常在编译时不能被简单地忽略。
- 运行时异常：运行时异常是可能被程序员避免的异常。与检查性异常相反，运行时异常可以在编译时被忽略。
- 错误：错误不是异常，而是脱离程序员控制的问题。错误在代码中通常被忽略。例如，当栈溢出时，一个错误就发生了，它们在编译也检查不到的。



捕获异常

异常捕获语句块的关键字如下：

- `try`：将被监听的代码放在 `try` 语句块之内，当语句块内发生异常时，异常就被抛出。
- `catch`：用来捕获 `try` 语句块中发生的异常。
- `finally`：`finally` 块的语句总是会被执行。主要用于回收在 `try` 块里打开的资源（如数据库连接、网络连接和磁盘文件）。只有 `finally` 块执行完成之后，才会回来执行 `try` 或者 `catch` 块中的 `return` 或者 `throw` 语句。如果 `finally` 中使用了 `return` 或者 `throw` 等终止方法的语句，则就不会跳回执行，直接停止。

异常捕获结构一般为 `try - catch - finally`，即：

```

try {
    ...
} catch (Exception1 e1) {
    ...
} catch (Exception2 e2) {
    ...
} finally {
    ...
}
  
```

其中 `catch` 和 `finally` 可以只出现一个。

抛出异常的关键字如下：

- `throws`：用于在方法的签名中声明该方法可能抛出的异常，可以一次抛出多个。
- `throw`：用于抛出异常。

自定义异常

自定义异常类时需注意：

- 所有异常类都必须是 `Throwable` 的子类。
- 如果希望自定义一个检查性异常类，则需要继承 `Exception` 类。
- 如果希望自定义一个运行时异常类，则需要继承 `RuntimeException` 类。

示例如下：

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class MyException extends Exception {
    public MyException() {
        super();
    }

    public MyException(String str) {
        super(str);
    }
}

public class example {
    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);

        System.out.println("请输入你的考试成绩: ");

        try {
            int score = input.nextInt();
            if (score < 0) {
                throw new MyException("成绩为负! ");
            } else if (score > 100) {
                throw new MyException("成绩超过阈值! ");
            } else {
                System.out.println("你的成绩为: "+score);
            }
        } catch (InputMismatchException e1) {
            System.out.println("输入的成绩不是数字! ");
        } catch (MyException e2) {
            System.out.println(e2.getMessage());
        }
    }
}
```

```
}  
}  
}
```

虚拟机

虚拟机是指通过软件模拟的具有完整硬件系统的，运行在一个完全隔离环境中的计算机系统。

Java Virtual Machine (JVM) 是通过软件来模拟 Java 字节码的指令集，是 Java 程序的运行环境。

特点：

1. 一次编译，到处运行
2. 自动管理内存
3. 自动回收垃圾

类与对象

对象

定义：客观存在的具体实体，具有明确定义的状态和行为

特征：标识符（区别其他对象）、属性（状态）、操作（行为）

- 属性：与对象关联的变量，描述对象的静态特性
- 操作：与对象相关联的函数，描述对象的动态特性

类

类与对象的关系

- 类是对象的抽象，是创建对象的模板
 - 对象是类的具体事例
 - 同一个类可以定义多个对象
- 二者比较
1. 类是静态的，在程序设计的时候就定义好了
 2. 对象是动态的，在程序执行的时候可以被创建、修改、删除

类的构造

类的访问权限：

	private	default	protected	public
同一个类中	√	√	√	√
同一个包中		√	√	√
子类中			√	√
全局范围内				√

下面通过代码解释：

```
java
├─ pkg1
│   ├─ Base.java
│   └─ SamePkgTest.java
└─ pkg2
    ├─ Sub.java
    └─ Other.java
```

```
/* pkg1 */
package pkg1;

public class Base {
    private Base() { System.out.println("private Base()"); }
    /* default */ Base(int x) { System.out.println("default Base()"); }
    protected Base(String s) { System.out.println("protected Base()"); }
}

public Base(double d) { System.out.println("public Base()"); }

public class SamePkgTest {
    public static void main(String[] args) {
        new Base();           // ✓ 同包可访问 private? -- 编译失败
        new Base(1);          // ✓ 同包 default 可见
        new Base("hi");       // ✓ 同包 protected 可见
        new Base(3.14);       // ✓ 同包 public 可见
    }
}
```

```
/* pkg2 */
package pkg2;
import pkg1.Base;

public class Sub extends Base {
    public Sub() {
        super();              // ✗ private 构造器，任何外类都不可见
    }
}
```



```

        super(1);           // ❌ default 构造器，异包不可见
        super("hello");     // ✅ protected 构造器，子类可见
        super(3.14);       // ✅ public 构造器永远可见
    }

    public static void main(String[] args) {
        new Sub();
    }
}

public class Other {
    public static void main(String[] args) {
        new Base();         // ❌ private
        new Base(1);        // ❌ default
        new Base("hi");     // ❌ protected
        new Base(2.71);     // ✅ 只有 public 对外可见
    }
}

```

静态类

静态成员（静态属性或静态方法）属于类所有，而不是属于某一具体对象所有。

加载时间

1. 静态成员在类加载的时候被静态地分配地址空间和方法的入口地址
2. 静态属性当且仅当在类初次加载的时候初始化

实例

1. 非静态对象在创建对象的时候初始化，存在多个实例副本，各个对象之间的副本不互相影响
2. 静态对象被所有的对象共享，在内存中只有一个副本

调用限制

1. `static` 方法不可访问非 `static` 的方法：非 `static` 的对象必须要实例化才能访问调用
2. 非 `static` 的方法可以访问 `static` 的方法：通过类名访问

静态块

1. 可以放在类的任何地方，类中可以有多个 `static` 块
2. 生命周期：在类加载的时候执行且只执行一次
3. 用法：用于初始化静态变量和调用静态方法
4. 语法：

```

class Person {
    static {

```

```
        totalNum = 10000;
        System.out.println("static block run!");
    }
}
```

静态是否破坏了面向对象的特性？

1. 静态属于类而非具体对象
2. 具有一定程度上的全局性：初始化时加载到内存，并且所有对象有访问权限
3. 保持类的封装性

继承

使用关键词 `extends` 从一个已存在的类派生出正在构造的新类。

- 子类不能直接继承父类的构造方法，需利用 `super` 关键字。
- 子类能复用父类的属性和方法，子类能增加一些新的属性与方法。

这里以 `Student`（子类）和 `Person`（父类）为例

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

public class Student extends Person {
    String school;

    public Student(String name, int age, String school) {
        super(name, age);
        this.school = school;
    }

    public String getSchool() {
        return this.school;
    }
}
```

```
}  
}
```

超类

`Object` 类是 `Java` 中所有类的始祖，`Java` 中每个类都扩展了 `Object`。因此 `Java` 中的所有类都可以使用 `Object` 的所有方法，如 `equals()`，用于比较两个对象是否相等（两个对象引用是否指向的是同一个对象）。

```
class IfStudentEqual {  
    public static void main(String[] args) {  
        Object Student_1 = new Student("Rick",19);  
        Object Student_2 = new Student("Morty",18);  
  
        // 不同对象，内存地址不同，不相等，返回 false  
        System.out.println(Student_1.equals(Student_2));  
  
        // 对象引用，内存地址相同，相等，返回 true  
        Object Student_3 = Student_1;  
        System.out.println(Student_1.equals(Student_3));  
    }  
}
```

`super` 关键字的功能：

- 在子类的构造方法中显式地调用父类构造方法。
- 访问父类的成员方法和变量。

抽象类与接口

父类是将子类所共同拥有的属性和方法进行抽取。如果有些属性和方法还无法确定，那么我们就可以将其定义成抽象方法，在后面子类进行具体化实现。拥有抽象方法的类就是抽象类，其无法被实例化。

```
public abstract class Person {  
    // Normal method  
    public String getName() {  
        return name;  
    }  
  
    // Abstract method  
    public abstract void getMission();  
}  
  
public class Teacher extends Person {  
    public void getMission() {  
        System.out.println("立德树人");  
    }  
}
```

```
}  
}
```

接口 (**interface**): 没有字段, 所有方法都是抽象方法。

当一个具体的类去实现一个 `interface` 时, 使用 `implements` 关键字

```
public interface Person {  
    void getDuty();  
    void getMission();  
}  
  
public class Teacher implements Person {  
    private String name;  
    private int age;  
  
    @Override  
    public void getDuty() {  
        System.out.println("教书育人");  
    }  
  
    @Override  
    public void getMission() {  
        System.out.println("立德树人");  
    }  
}
```

抽象类与接口的不同之处在于, 当一个类继承抽象类时, 它只能 `extends` 一个抽象类; 而当一个类来实现接口时, 它可以一次 `implements` 多个 `interface`。

多重继承

多重继承 (**multiple inheritance**) 指的是一个类可以同时从多于一个父类继承属性与方法。多重继承存在问题:

- 若继承的父类拥有相同的成员变量, 子类在引用该变量时将无法判别使用哪个父类的成员变量。
- 同理, 若继承的父类拥有相同的方法签名, 同时子类并未覆盖该方法 (若覆盖则直接使用子类的实现), 那么调用该方法时将无法确定调用哪个父类的方法。

使用 内部类 实现多重继承如下:

```
public class Father {  
    public int strong() {  
        return 9;  
    }  
}
```

```

public class Mother {
    public int smart() {
        return 8;
    }
}

public class Son {
    class FatherInner extends Father {
        public int strong() {
            return super.strong() + 1;
        }
    }

    class MotherInner extends Mother {
        public int smart() {
            return super.smart() + 1;
        }
    }

    public int getStrong() {
        return new FatherInner.strong();
    }

    public int getSmart() {
        return new MotherInner.smart();
    }
}

```

使用 接口 实现多重继承如下：

```

public interface Father {
    public void strong();
}

public interface Mother {
    public void smart();
}

public interface Son extends Father, Mother {
    public void kind();
}

public class Boy implements Son {
    @Override
    public void strong() {
        System.out.println("He's strong.")
    }

    @Override

```

```
public void smart() {  
    System.out.println("He's smart.")  
}  
  
@Override  
public void kind() {  
    System.out.println("He's kind.")  
}  
}
```